

OBJECT-ORIENTED PROGRAMMING
WEEK 1 LAB — WORKED SOLUTION

Exercises 1-3 and the Week 1 Lab Assignment (Part A & Part B), completed

Student Name	Daniyal Mushtaq	Seat No.	B25110006040
Section	[] A [✓] B [] C	Date	25 July 2026

This document works through every deliverable listed in the manual: the five-favourite-objects worksheet, the annotated Stack/Heap diagram for the Car demo, and the enhanced **Car** and **Calculator** classes required by the Week 1 Lab Assignment (Part A and Part B), each with an updated UML diagram, full source code, a driver (Main.java) and its expected console output.

1 Activity 1 — Five Favourite Objects
Attributes/behaviours worksheet

#	Favourite object	Relevant attributes (what it has)	Behaviours (what it can do)
1	Smartphone	brand, model, batteryLevel, screenOn, volume	powerOn(), powerOff(), makeCall(), sendText(), adjustVolume()
2	Backpack	capacityLitres, color, numPockets, currentLoad	open(), close(), packItem(), removeItem(), zipUp()
3	Bicycle	brand, gear, speed, wheelSize, isLocked	pedal(), brake(), changeGear(), lock(), unlock()
4	Wristwatch	brand, time, isWaterproof, batteryLevel	setTime(), startStopwatch(), stopStopwatch(), setAlarm()
5	Headphones	brand, volumeLevel, isConnected, batteryLevel	connect(), disconnect(), increaseVolume(), decreaseVolume(), mute()

2 Exercise 2 — Stack and Heap for the Car Demo
Where reference variables and objects actually live

Two reference variables, `bmw` and `tesla`, are declared inside `main()`. Each new `Car()` call creates a separate object on the heap; the stack only ever holds the address (reference) of that object — never the object itself.

```
Car bmw = new Car();      Car tesla = new Car();
```

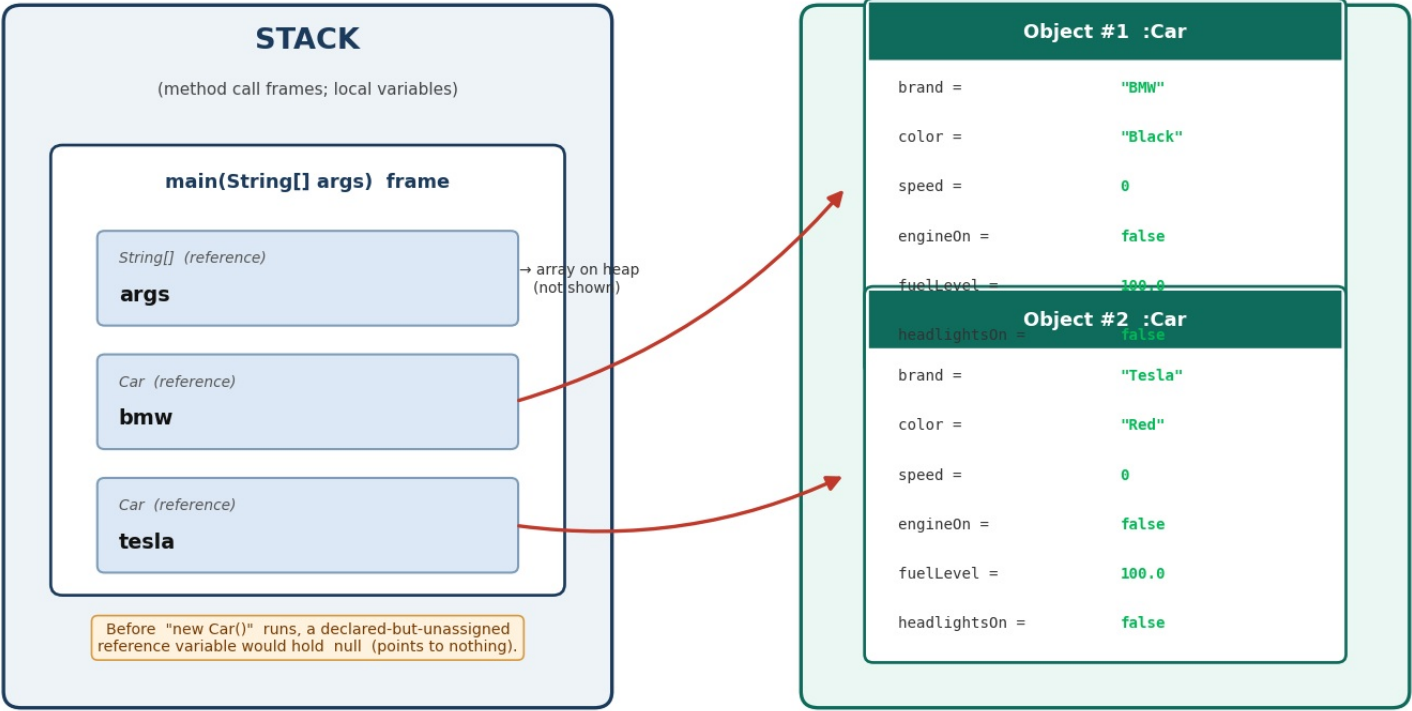


Figure E2.1 — Stack frame for `main()` holding two references, each pointing to its own `Car` object on the heap.

Listing E2 — Main.java (Car demo, annotated)

```
/*
 * File: Main.java
 * Purpose: Creates and controls Car objects.
 * Note: bmw and tesla are STACK reference variables.
 *       Until "new Car()" runs, an uninitialized reference
 *       would hold null (it would point to no object at all).
 */
public class Main {
    public static void main(String[] args) {
        // 1) "new Car()" allocates a Car object on the HEAP and runs its constructor.
        // 2) The address of that new object is copied into the stack variable "bmw".
        Car bmw = new Car();
        // A second, completely separate object is created for "tesla".
        // bmw and tesla now reference two different heap objects (see Figure E2.1).
        Car tesla = new Car();

        bmw.setDetails("BMW", "Black");
        tesla.setDetails("Tesla", "Red");

        bmw.startEngine();
        tesla.startEngine();

        // Calling displayState() through each reference reads that
        // object's own fields on the heap -- changing bmw never touches tesla.
        bmw.displayState();
        tesla.displayState();

        bmw.stopEngine();
        bmw.displayState();
    }
}
```

Exercise 3 (varargs add): the add(double... numbers) method from the manual is implemented and demonstrated directly inside the enhanced **Calculator** class in Part B below, alongside the other new operations.

Updated UML diagram (drawn before editing Car.java)

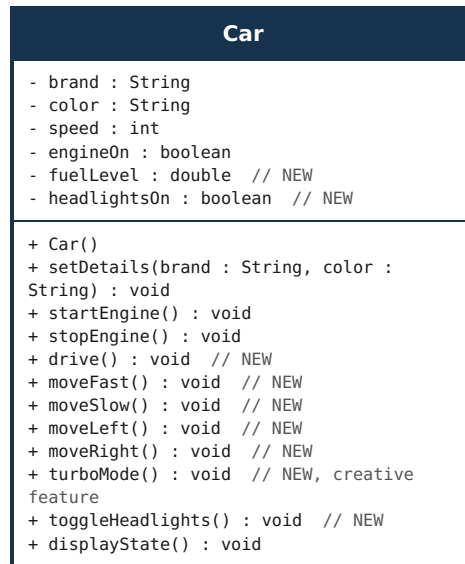


Figure A.1 — Updated UML for Car: two new attributes, six new behaviours (one creative).

Rules enforced:

- Every movement/turbo method refuses to act while engineOn is false.
- moveSlow() clamps speed at 0 — it can never go negative.
- stopEngine() resets speed to 0 and turns the engine off.
- turboMode() additionally refuses to engage if fuelLevel is below 10%.

Listing A.1 — Car.java (enhanced)

```
public class Car {
    // ---- original attributes ----
    private String brand;
    private String color;
    private int speed;
    private boolean engineOn;

    // ---- new attributes (Part A, step 2) ----
    private double fuelLevel;    // percentage, 0-100
    private boolean headlightsOn;

    public static final int MAX_SPEED = 220;    // km/h reached by turboMode()
    public static final int SPEED_STEP = 10;    // km/h changed per moveFast/moveSlow

    public Car() {
        this.brand = "Unknown";
        this.color = "Unpainted";
        this.speed = 0;
        this.engineOn = false;
        this.fuelLevel = 100.0;
        this.headlightsOn = false;
    }

    public void setDetails(String brand, String color) {
        this.brand = brand;
        this.color = color;
    }

    public void startEngine() {
        if (this.fuelLevel <= 0) {
            System.out.println(this.brand + " cannot start: fuel tank empty.");
            return;
        }
        this.engineOn = true;
        System.out.println(this.brand + " engine started.");
    }

    public void stopEngine() {
        this.speed = 0;                // rule: stopping resets speed to zero
        this.engineOn = false;
        System.out.println(this.brand + " engine stopped.");
    }

    // ----- movement behaviours -----
    public void drive() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot drive: engine is off.");
            return;
        }
    }
}
```

```

        this.speed = 40;
        consumeFuel(2.0);
        System.out.println(this.brand + " is now driving at " + this.speed + " km/h.");
    }

    public void moveFast() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot accelerate: engine is off.");
            return;
        }
        this.speed += SPEED_STEP;
        consumeFuel(1.5);
        System.out.println(this.brand + " speeds up to " + this.speed + " km/h.");
    }

    public void moveSlow() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot slow down: engine is off.");
            return;
        }
        this.speed -= SPEED_STEP;
        if (this.speed < 0) {
            this.speed = 0;           // rule: speed must never become negative
        }
        System.out.println(this.brand + " slows down to " + this.speed + " km/h.");
    }

    public void moveLeft() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot turn: engine is off.");
            return;
        }
        System.out.println(this.brand + " turns left.");
    }

    public void moveRight() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot turn: engine is off.");
            return;
        }
        System.out.println(this.brand + " turns right.");
    }

    // ----- creative / thrilling feature -----
    public void turboMode() {
        if (!this.engineOn) {
            System.out.println(this.brand + " cannot engage turbo: engine is off.");
            return;
        }
        if (this.fuelLevel < 10) {
            System.out.println(this.brand + " turbo refused: fuel too low.");
            return;
        }
        this.speed = MAX_SPEED;
        consumeFuel(15.0);
        System.out.println(this.brand + " engages TURBO MODE! Speed: " + this.speed + " km/h.");
    }

    public void toggleHeadlights() {
        this.headlightsOn = !this.headlightsOn;
        System.out.println(this.brand + " headlights " + (this.headlightsOn ? "ON" : "OFF") + ".");
    }

    private void consumeFuel(double amount) {
        this.fuelLevel -= amount;
        if (this.fuelLevel < 0) {
            this.fuelLevel = 0;
        }
    }

    public void displayState() {
        System.out.println("Brand      : " + this.brand);
        System.out.println("Color      : " + this.color);
        System.out.println("Speed      : " + this.speed + " km/h");
        System.out.println("Engine on   : " + this.engineOn);
        System.out.println("Fuel level  : " + this.fuelLevel + " %");
        System.out.println("Headlights  : " + (this.headlightsOn ? "ON" : "OFF"));
        System.out.println();
    }
}

```

Listing A.2 — Main.java (Car demo — Part A)

```
public class Main {
    public static void main(String[] args) {
        Car bmw = new Car();
        Car tesla = new Car();

        bmw.setDetails("BMW", "Black");
        tesla.setDetails("Tesla", "Red");

        System.out.println("--- Trying to drive before starting the engine ---");
        bmw.drive();

        System.out.println("\n--- Starting engines ---");
        bmw.startEngine();
        tesla.startEngine();

        System.out.println("\n--- BMW driving sequence ---");
        bmw.drive();
        bmw.moveFast();
        bmw.moveFast();
        bmw.moveLeft();
        bmw.toggleHeadlights();
        bmw.turboMode();

        System.out.println("\n--- Tesla driving sequence ---");
        tesla.drive();
        tesla.moveSlow();
        tesla.moveSlow();
        tesla.moveSlow();
        tesla.moveSlow();
        tesla.moveSlow(); // speed already 0 -- demonstrates the "never negative" rule
        tesla.moveRight();

        System.out.println("\n--- States after independent actions ---");
        bmw.displayState();
        tesla.displayState();

        System.out.println("--- Stopping BMW ---");
        bmw.stopEngine();
        bmw.displayState();
    }
}
```

Console output — Listing A.2

```
--- Trying to drive before starting the engine ---
BMW cannot drive: engine is off.
```

```
--- Starting engines ---
BMW engine started.
Tesla engine started.
```

```
--- BMW driving sequence ---
BMW is now driving at 40 km/h.
BMW speeds up to 50 km/h.
BMW speeds up to 60 km/h.
BMW turns left.
BMW headlights ON.
BMW engages TURBO MODE! Speed: 220 km/h.
```

```
--- Tesla driving sequence ---
Tesla is now driving at 40 km/h.
Tesla slows down to 30 km/h.
Tesla slows down to 20 km/h.
Tesla slows down to 10 km/h.
Tesla slows down to 0 km/h.
Tesla slows down to 0 km/h.
Tesla turns right.
```

```
--- States after independent actions ---
Brand      : BMW
Color      : Black
Speed      : 220 km/h
Engine on  : true
Fuel level : 80.0 %
Headlights : ON
```

```
Brand      : Tesla
Color      : Red
Speed      : 0 km/h
Engine on  : true
Fuel level : 98.0 %
Headlights : OFF
```

```
--- Stopping BMW ---
BMW engine stopped.
Brand      : BMW
Color      : Black
Speed      : 0 km/h
Engine on  : false
Fuel level : 80.0 %
Headlights : ON
```


Updated UML diagram (drawn before coding)

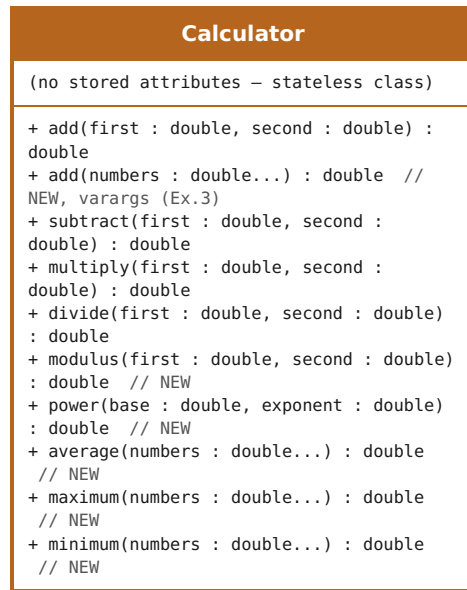


Figure B.1 — Updated UML for Calculator: five new operations plus the varargs add().

Invalid input handling: divide() and modulus() throw an ArithmeticException when the second operand is 0, instead of silently returning a wrong number. average(), maximum() and minimum() throw an IllegalArgumentException when called with no operands.

Listing B.1 — Calculator.java (enhanced)

```
public class Calculator {

    public double add(double first, double second) {
        return first + second;
    }

    // Exercise 3: varargs version -- accepts any number of operands.
    public double add(double... numbers) {
        double total = 0;
        for (double number : numbers) {
            total += number;
        }
        return total;
    }

    public double subtract(double first, double second) {
        return first - second;
    }

    public double multiply(double first, double second) {
        return first * second;
    }

    public double divide(double first, double second) {
        if (second == 0) {
            throw new ArithmeticException("Division by zero is not allowed.");
        }
        return first / second;
    }

    // ----- new operations (Part B, step 2) -----
    public double modulus(double first, double second) {
        if (second == 0) {
            throw new ArithmeticException("Modulus by zero is not allowed.");
        }
        return first % second;
    }

    public double power(double base, double exponent) {
        return Math.pow(base, exponent);
    }

    public double average(double... numbers) {
        if (numbers.length == 0) {
            throw new IllegalArgumentException("Cannot average zero values.");
        }
        return add(numbers) / numbers.length;
    }

    public double maximum(double... numbers) {
        if (numbers.length == 0) {
            throw new IllegalArgumentException("Cannot find maximum of zero values.");
        }
        double max = numbers[0];
        for (double n : numbers) {
            if (n > max) { max = n; }
        }
        return max;
    }

    public double minimum(double... numbers) {
        if (numbers.length == 0) {
            throw new IllegalArgumentException("Cannot find minimum of zero values.");
        }
        double min = numbers[0];
        for (double n : numbers) {
            if (n < min) { min = n; }
        }
        return min;
    }

}
```

Listing B.2 — Main.java (Calculator demo — Part B)

```
public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("--- add / subtract / multiply ---");
        System.out.println("add(4, 7)      = " + calc.add(4, 7));
        System.out.println("subtract(10, 3) = " + calc.subtract(10, 3));
        System.out.println("multiply(6, 5)  = " + calc.multiply(6, 5));

        System.out.println("\n--- divide: normal and edge case ---");
        System.out.println("divide(20, 4)   = " + calc.divide(20, 4));
        try {
            System.out.println("divide(5, 0)    = " + calc.divide(5, 0));
        } catch (ArithmeticException e) {
            System.out.println("divide(5, 0)    -> Error: " + e.getMessage());
        }

        System.out.println("\n--- modulus: normal and edge case ---");
        System.out.println("modulus(17, 5)  = " + calc.modulus(17, 5));
        try {
            System.out.println("modulus(9, 0)   = " + calc.modulus(9, 0));
        } catch (ArithmeticException e) {
            System.out.println("modulus(9, 0)   -> Error: " + e.getMessage());
        }

        System.out.println("\n--- power: normal and edge case ---");
        System.out.println("power(2, 10)    = " + calc.power(2, 10));
        System.out.println("power(9, 0.5)  = " + calc.power(9, 0.5));

        System.out.println("\n--- average / maximum / minimum ---");
        System.out.println("average(4,8,15) = " + calc.average(4, 8, 15));
        System.out.println("maximum(4,8,15) = " + calc.maximum(4, 8, 15));
        System.out.println("minimum(4,8,15) = " + calc.minimum(4, 8, 15));
        try {
            System.out.println("average()      = " + calc.average());
        } catch (IllegalArgumentException e) {
            System.out.println("average()      -> Error: " + e.getMessage());
        }

        System.out.println("\n--- varargs add: any number of operands ---");
        System.out.println("add(1,2,3,4,5) = " + calc.add(1, 2, 3, 4, 5));
    }
}
```

Console output — Listing B.2

```
--- add / subtract / multiply ---
add(4, 7)      = 11.0
subtract(10, 3) = 7.0
multiply(6, 5)  = 30.0

--- divide: normal and edge case ---
divide(20, 4)   = 5.0
divide(5, 0)    -> Error: Division by zero is not allowed.

--- modulus: normal and edge case ---
modulus(17, 5)  = 2.0
modulus(9, 0)   -> Error: Modulus by zero is not allowed.

--- power: normal and edge case ---
power(2, 10)    = 1024.0
power(9, 0.5)   = 3.0

--- average / maximum / minimum ---
average(4,8,15) = 9.0
maximum(4,8,15) = 15.0
minimum(4,8,15) = 4.0
average()       -> Error: Cannot average zero values.

--- varargs add: any number of operands ---
add(1,2,3,4,5) = 15.0
```

Recommended folder layout for submission (matches the manual's Section 10):

```
week01_<name>_<last 3 digits of seat No>/
  CarDemo/   Car.java, Main.java, Car_UML.png
  CalculatorDemo/ Calculator.java, Main.java, Calculator_UML.png
  SampleOutput.txt or screenshots/
```

Compile and run each demo from inside its own folder: `javac Car.java Main.java` then `java Main` (and the same pattern for `CalculatorDemo`).